অধ্যায় ১৫

JavaScript Browser APIs, Web Storage, Fetch, URL এবং Real-World Browser Applications

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Browser API কীভাবে কাজ করে, `window` ও `document`-এর দায়িত্ব, URL API, Web Storage, Fetch, AbortController, File/Blob API এবং browser security boundary বুঝতে পারবে। শেষে **Bookmark Vault** নামে একটি বাস্তব Browser Application তৈরি করবে, যেখানে Search, Tag Filter, Favorites, Import/Export এবং Persistent Storage থাকবে।

১৫.১ Browser API কী এবং JavaScript Language থেকে পার্থক্য

JavaScript ভাষা `Array`, `Object`, `Promise`, `Map` ইত্যাদি দেয়। Browser সেই ভাষার বাইরে `document`, `localStorage`, `fetch`, `URL`, `navigator`, `history`, `location`, `Blob` ইত্যাদি Host APIs দেয়। Node.js-এ একই Browser DOM API সাধারণত পাওয়া যায় না।

```javascript
console.log(typeof Promise);      // function
console.log(window.location.href); // বর্তমান পেজের URL
console.log(document.title);       // বর্তমান document title
```

Browser APIs-এর কিছু অংশ secure context (HTTPS বা localhost) দাবি করে। API availability যাচাই করা ভালো:

```javascript
if ('clipboard' in navigator) {
  // Clipboard API সম্ভবত উপলব্ধ; permission লাগতে পারে
}
```

`window` হলো browser window-এর global object; `document` হলো DOM tree-এর entry point। `location` current URL নির্দেশ করে। Browser এবং Server-এর JavaScript পরিবেশ আলাদা হওয়ায় runtime capability ধরে না নিয়ে পরীক্ষা করা উচিত।

১৫.২ URL এবং URLSearchParams

String জোড়া লাগিয়ে URL বানালে encoding, query separator, whitespace ইত্যাদি ভুল হতে পারে। `URL` ও `URLSearchParams` এই সমস্যা কমায়।

```javascript
const url = new URL('https://example.com/search');
url.searchParams.set('q', 'বাংলা JavaScript');
url.searchParams.set('page', '2');
console.log(url.href);
console.log(url.searchParams.get('q'));
```

`URL`-এর `protocol`, `hostname`, `pathname`, `searchParams`, `hash` ইত্যাদি property রয়েছে। `new URL('/help', 'https://example.com/docs/')` দিয়ে relative path resolve করা যায়। তবে **সঠিকভাবে parse হওয়া মানেই URL নিরাপদ নয়**: `javascript:` বা `data:` scheme অনিচ্ছাকৃতভাবে গ্রহণ করা বিপজ্জনক। Bookmark-এর জন্য আমরা কেবল `http:` এবং `https:` গ্রহণ করব।

```javascript
function safeWebUrl(value) {
  const url = new URL(value);
  if (!['http:', 'https:'].includes(url.protocol)) {
    throw new Error('Unsupported URL scheme');
  }
  return url.href;
}
```

১৫.৩ Web Storage: localStorage ও sessionStorage

`localStorage`-এর data একই origin-এর browser storage-এ সাধারণত session পেরিয়েও থাকে; `sessionStorage` tab-এর session-এ সীমাবদ্ধ। উভয় API key-value string store করে।

```javascript
localStorage.setItem('theme', 'dark');
console.log(localStorage.getItem('theme')); // dark
localStorage.removeItem('theme');

const preferences = {language: 'bn', compact: true};
localStorage.setItem('prefs', JSON.stringify(preferences));
const saved = JSON.parse(localStorage.getItem('prefs') ?? '{}');
```

সীমাবদ্ধতা: Web Storage synchronous; বড় data লিখলে UI block হতে পারে। Browser quota, privacy settings, private mode অথবা sandbox policy-র কারণে operation ব্যর্থ হতে পারে। তাই `try/catch` ব্যবহার করো। `file://` থেকে Storage-এর আচরণ browserভেদে ভিন্ন হতে পারে; local server দিয়ে চালানো ভালো।

```javascript
function persist(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
    return true;
  } catch (error) {
    console.warn('Save failed:', error);
    return false;
  }
}
```

Security: Password, API Secret, Access Token-এর মতো Sensitive Data localStorage-এ রাখাকে নিরাপদ default ধরা যাবে না। XSS হলে page-এর JavaScript Storage পড়তে পারে। এই Chapter-এর Bookmark Data ব্যক্তিগত হলেও high-sensitivity secret নয়; ব্যবহারকারীকে Export File নিরাপদে রাখতে হবে।

১৫.৪ JSON Serialization এবং Data Validation

`JSON.stringify()` Object-কে String করে; `JSON.parse()` String থেকে Value তৈরি করে। কিন্তু Parsed Data কখনোই স্বয়ংক্রিয়ভাবে trustworthy নয়।

```javascript
const raw = localStorage.getItem('bookmarks');
let bookmarks = [];
try {
  const parsed = raw ? JSON.parse(raw) : [];
  if (Array.isArray(parsed)) bookmarks = parsed;
} catch {
  bookmarks = [];
}
```

`JSON.stringify()` Function ও `undefined`-এর মতো কিছু value বাদ দিতে পারে, `BigInt` সরাসরি serialize করা যায় না, এবং circular reference-এ Error হয়। Stored Schema বদলালে Migration Plan দরকার—যেমন `version: 1`।

১৫.৫ Fetch API: HTTP Request ও Response

`fetch()` Promise return করে। Network Error হলে Promise reject হতে পারে, কিন্তু HTTP 404 বা 500 হলে সাধারণত `fetch` reject করে না; `response.ok` পরীক্ষা করতে হয়।

```javascript
async function loadPosts() {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts?_limit=3');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return await response.json();
}

loadPosts().then(console.log).catch(console.error);
```

`response.json()`-ও ব্যর্থ হতে পারে যদি response JSON না হয়। API-র response data ব্যবহার করার আগে structure validate করো। `POST` request-এ `method`, `headers` ও `body` প্রয়োজন হতে পারে:

```javascript
const payload = {title: 'New note'};
const response = await fetch('/api/notes', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify(payload)
});
if (!response.ok) throw new Error('Request failed');
```

CORS: অন্য origin-এ Browser Request করলে Server-এর CORS Policy প্রযোজ্য হয়। Frontend থেকে ইচ্ছেমতো Header যোগ করে Server-এর CORS restriction bypass করা যায় না। Authentication ও authorization Server-এ enforce করতে হয়।

১৫.৬ AbortController ও Request Cancellation

Search Box-এ দ্রুত টাইপ করলে পুরোনো request পরে শেষ হয়ে নতুন result overwrite করতে পারে। `AbortController` দিয়ে আগের request cancel করা যায়।

```javascript
let controller;
async function searchRemote(query) {
  controller?.abort();
  controller = new AbortController();
  try {
    const url = new URL('/api/search', location.origin);
    url.searchParams.set('q', query);
    const response = await fetch(url, {signal: controller.signal});
    if (!response.ok) throw new Error('Search failed');
    return await response.json();
  } catch (error) {
    if (error.name === 'AbortError') return null;
    throw error;
  }
}
```

Request cancel হলেও Server-এ ইতিমধ্যে শুরু হওয়া কাজ সম্পূর্ণ বন্ধ হয়েছে—এমন নিশ্চয়তা নেই। Race Condition-এর জন্য Request ID/Latest-Response Check-ও ব্যবহার করা যায়।

১৫.৭ Browser Events এবং Cross-Tab Storage

`addEventListener()` দিয়ে Event Handler register করা হয়। `storage` event সাধারণত একই origin-এর **অন্য** document-এ Storage বদলালে পাওয়া যায়; যে document নিজে পরিবর্তন করেছে সেখানে এই event trigger হয় না।

```javascript
window.addEventListener('storage', event => {
  if (event.key === 'jsmastery.bookmarks.v1') {
    console.log('অন্য tab-এ Bookmark data বদলেছে');
  }
});
```

Production App-এ অন্য Tab-এর পরিবর্তন এলে UI reload বা state merge করার ব্যবস্থা করা যায়। আমাদের ছোট Project-এ cross-tab live synchronization রাখা হয়নি—এটি Extension Exercise।

১৫.৮ File, Blob এবং Download API

Client-side JSON Export করতে Server প্রয়োজন নেই। `Blob` দিয়ে File Content তৈরি করে Object URL-এর মাধ্যমে Download করা যায়।

```javascript
const blob = new Blob([JSON.stringify({hello: 'world'}, null, 2)],
  {type: 'application/json'});
const objectUrl = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = objectUrl;
a.download = 'example.json';
a.click();
URL.revokeObjectURL(objectUrl);
```

File Input থেকে `file.text()` দিয়ে User-selected File পড়া যায়। Import-এর আগে File Size, Format ও Data Schema validate করতে হবে। JSON-এর মধ্যে HTML বা URL থাকলে তা `innerHTML`-এ বসানো যাবে না।

১৫.৯ Practical Project — Bookmark Vault

সমস্যা ও Requirements

অনেক Developer Documentation, Tutorials ও Tools-এর URL আলাদা জায়গায় রেখে পরে খুঁজে পান না। **Bookmark Vault**-এ আমরা URL সংরক্ষণ, Tagging, Search, Favorites, Delete, JSON Export/Import এবং Local Persistence যুক্ত করব। Backend বা Account ছাড়া একটি Local-first Learning App হবে।

Project Folder:

```
source-code/chapter-15/
├── index.html
├── style.css
├── bookmarks.mjs
├── app.mjs
└── bookmarks.test.mjs
```

`bookmarks.mjs`-এ pure validation, filtering ও import/export function; `app.mjs`-এ DOM এবং Storage orchestration। এভাবে UI Logic ও Business Logic আলাদা থাকায় Unit Test সহজ হয়।

Step 1 — Data Model

```javascript
{
  id: 'generated-unique-id',
  title: 'MDN JavaScript',
  url: 'https://developer.mozilla.org/',
  tags: ['javascript', 'reference'],
  createdAt: '2026-10-11T00:00:00.000Z',
  favorite: false
}
```

`id` হলো Identity, `url` হলো Validated Address, `tags` হলো Search/Filter Metadata। `createdAt` ISO Timestamp; এটি Client Clock-এর উপর নির্ভরশীল।

Step 2 — Validation এবং Normalization

`validateBookmark()` Title Trim করে, URL Protocol Check করে, Tags Normalize করে। `createBookmark()` Unique ID ও Timestamp যুক্ত করে। Full implementation `bookmarks.mjs`-এ রয়েছে।

```javascript
const clean = validateBookmark({
  title: '  MDN  ',
  url: 'https://developer.mozilla.org/#docs',
  tags: 'JavaScript, Reference, javascript'
});
// title: 'MDN'; hash বাদ; tags duplicate বাদ
```

Step 3 — UI Event এবং Rendering

`app.mjs` Form Submit intercept করে, Bookmark Add করে, তারপর `render()` চালায়। `render()` DOM Element তৈরি করে `textContent` ব্যবহার করে—User Input HTML হিসেবে execute হয় না। Link-এর `target="_blank"`-এর সঙ্গে `rel="noopener noreferrer"` রাখা হয়েছে।

Step 4 — Storage

```javascript
const items = safeParseBookmarks(localStorage.getItem(STORAGE_KEY));
localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
```

Import-এ Duplicate URL বাদ দেওয়া হয়। Export JSON-এ `version` Field থাকে, যাতে ভবিষ্যতে Schema Upgrade সহজ হয়। এই Demo-তে Storage Write Fail করলে Message দেখানো হয়; Production-এ Retry/Backup Strategy লাগতে পারে।

Step 5 — App চালানো

```terminal
cd source-code/chapter-15
python -m http.server 8000
```

তারপর Browser-এ `http://localhost:8000` খুলো। Project `type="module"` ব্যবহার করে, তাই `file://`-এর পরিবর্তে Local Server সুপারিশ করা হচ্ছে। Node.js থাকলে Test চালাও:

```terminal
node --test bookmarks.test.mjs
```

Manual Acceptance Checklist

- ☐ Valid HTTPS URL Add করলে List-এ দেখা যায়
- ☐ `javascript:` URL Reject হয়

☐ Duplicate URL Add করা যায় না

☐ Search ও Tag Filter কাজ করে

☐ Favorite Toggle এবং Delete কাজ করে

☐ Refresh-এর পর Saved Data থাকে

☐ Export → Import করলে Bookmark ফিরে আসে

☐ Corrupt JSON Import করলে Error Message দেখায়

☐ Mobile Width-এ Layout ব্যবহারযোগ্য থাকে

১৫.১০ নিরাপত্তা, Privacy ও Architecture Review

1. **XSS:** User Input Render করার সময় `textContent` ব্যবহার করো, `innerHTML` নয়।
2. **URL Validation:** `http:` / `https:` ছাড়া Protocol Reject করো। এটি সমস্ত Phishing/Unsafe Destination আটকায় না।
3. **LocalStorage:** Same-Origin Scripts Data পড়তে পারে। Confidential Credential রাখবে না।
4. **Import Trust:** File Extension `.json` হলেও Content যাচাই প্রয়োজন।
5. **Network:** `fetch` -এর Error ও HTTP Status আলাদা Handle করো।
6. **Accessibility:** Form Label, Focus State, Status Announcement (`aria-live`) গুরুত্বপূর্ণ।
7. **Data Integrity:** Import-এর আগে Backup Export রাখা ভালো; Real-world App-এ Schema Migration প্রয়োজন।
8. **Scalability:** হাজার হাজার Bookmark হলে IndexedDB, Pagination, Debounced Search বিবেচনা করো।

১৫.১১ Common Mistakes এবং Debugging

ভুল	সমস্যা	সমাধান
`JSON.parse(null)` ধরে নেওয়া	Unexpected State	Missing Value ও Schema Check
`fetch()` 404-এ Reject করবে ভাবা	Error UI কাজ করে না	`response.ok` Check
`innerHTML = title`	XSS Risk	`textContent`
`new URL()` সব Scheme Allow করা	Unsafe Links	Protocol Allowlist
`localStorage` Write কখনো Fail করবে না ভাবা	Data Loss	`try/catch` ও Feedback
Module `file://` থেকে চালানো	CORS/Module Error	Local HTTP Server
Storage Event একই Tab-এ আশা করা	UI Sync Bug	Same-tab State Update

১৫.১২ Exercises

1. `URLSearchParams` দিয়ে `category=js` এবং `page=3` Query String তৈরি করো।
2. `localStorage` -এ `theme` সংরক্ষণ এবং পড়ার Function লেখো।
3. HTTP Status 404 হলে Custom Error Throw করে এমন Fetch Wrapper লেখো।

4. URL Validator-এ `ftp:` Scheme Reject করো।
5. Bookmark Tags থেকে Unique Tags-এর Sorted List তৈরি করো।
6. `filterBookmarks()`-এ Case-insensitive Search পরীক্ষা করো।
7. Import-এর আগে 1MB File Size Limit যুক্ত করো।
8. Export JSON-এ `version` Field যোগ করো।
9. অন্য Tab-এ Storage বদলালে Current Tab-এর List Refresh করো।
10. Favorite Bookmarks আগে দেখানোর Sorting যুক্ত করো।

১৫.১৩ Sample Solutions

১. Query:

```javascript
const params = new URLSearchParams({category: 'js', page: '3'});
console.log(params.toString()); // category=js&page=3
```

২. Theme Storage:

```javascript
function saveTheme(theme) { localStorage.setItem('theme', theme); }
function getTheme() { return localStorage.getItem('theme') ?? 'light'; }
```

৩. Fetch Wrapper:

```javascript
async function getJSON(url) {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

৪. Protocol: `['http:', 'https:'].includes(new URL(input).protocol)` পরীক্ষা করো; False হলে Error Throw।

৫. Unique Tags:

```javascript
const tags = [...new Set(items.flatMap(x => x.tags))].sort();
```

৬. Search: `filterBookmarks(items, 'mdn')` ও `filterBookmarks(items, 'MDN')` একই ফল দেয় কি না Test করো।

৭. File Size: `if (file.size > 1024 * 1024) throw new Error('Too large')`।

৮. Versioned Export: `JSON.stringify({version: 1, bookmarks: items})`।

৯. Cross-tab:

```javascript
window.addEventListener('storage', event => {
  if (event.key === STORAGE_KEY) {
    items = safeParseBookmarks(event.newValue);
    render();
  }
});
```

১০. Favorite Sorting:

```javascript
const sorted = [...items].sort((a, b) => Number(b.favorite) - Number(a.favorite));
```

১৫.১৪ MCQ Quiz — ১০টি প্রশ্ন

১. `localStorage` কোন ধরনের Value Store করে? A) Object B) String C) Function D) DOM Node

২. `fetch()`-এ HTTP 404 এলে কী পরীক্ষা করা উচিত? A) `response.ok` B) `window.ok` C) `fetch.status` D) `JSON.ok`

৩. URL Query Parameter Encode করতে উপযোগী কোনটি? A) `URLSearchParams` B) `eval` C) `innerHTML` D) `setTimeout`

৪. Request Cancel করতে কোন API? A) `AbortController` B) `MutationObserver` C) `FileReader` D) `History`

৫. User-provided Title Render করতে কোনটি নিরাপদ Default? A) `innerHTML` B) `document.write` C) `textContent` D) `eval`

৬. JSON Text থেকে Value পেতে? A) `JSON.stringify` B) `JSON.parse` C) `JSON.load` D) `URL.parse`

৭. File Download-এর জন্য In-memory Content তৈরি করতে? A) `Blob` B) `Map` C) `WeakSet` D) `Proxy`

৮. `sessionStorage` সাধারণত কোন Scope-এ? A) সকল Browser B) Tab Session C) সব Device D) Server Database

৯. `storage` Event সাধারণত কোন Document-এ Fire হয়? A) যে Document পরিবর্তন করেছে B) অন্য Same-origin Document C) Server D) Service Worker মাত্র

১০. `javascript:` URL Allow করা উচিত? A) সবসময় B) Bookmark-এর জন্য না C) HTTPS হলে D) Only Chrome-এ

Answer Key ও ব্যাখ্যা

1. **B** — Storage Value String; Object হলে JSON Serialize করতে হয়।
2. **A** — 404/500-এ Promise সাধারণত Resolve হতে পারে; `ok` পরীক্ষা প্রয়োজন।
3. **A** — `URLSearchParams` Query Encode করে।
4. **A** — `AbortController`-এর `signal` Request-এ দেওয়া যায়।
5. **C** — `textContent` Input-কে Text হিসেবে Render করে।
6. **B** — `JSON.parse()` JSON Text Parse করে।
7. **A** — `Blob` File-like Data Represent করে।
8. **B** — `sessionStorage` Tab Session-ভিত্তিক।

9. **B** — সাধারণত অন্য Same-origin Browsing Context-এ।
10. **B** — Untrusted Bookmark URL-এ Unsafe Scheme Reject করতে হয়।

১৫.১৫ Chapter Summary ও Next Step

এই অধ্যায়ে Browser Environment, URL Processing, Storage, Fetch, Cancellation, File Import/Export, Safe Rendering এবং Validation শিখেছ। Bookmark Vault-এর Source Code ও Tests Chapter Folder-এ রয়েছে। পরের অধ্যায়ে আমরা **Error Handling, Debugging, Testing এবং Software Reliability**-তে এগোব, যেখানে Error Scenario-কে Testable Behavior-এ রূপান্তর করব।